

```
import { goldPlus, nimm, probe, schaden } from
"./engine";
import { vielleichtHeiltrank } from "./heal";
import type { Runtime } from "./runtime";
import { HEILTRANK, LEICHT, MITTEL, SCHWER, tot,
type Held } from "./types";

export async function dorfTruebesWasser(rt:
Runtime, held: Held) {
  if (held.loesungswegBrunnen) {
    await brunnenNachspiel(rt, held);
    return;
  }

  await rt.present({
    title: "Der bittere Krug",
    art: "well",
    portrait: null,
    held,
    lines: [
      "Der Wassereimer am Dorfbrunnen steht halb
voll. Niemand hat ihn heute Morgen geleert.",
      "Ein Kind hustet vor der Apotheke. Die
Mutter hält es fester, als das Husten es
verlangt.",
      "Am Brunnenrand steht Ratsherr Dennek und
rührt mit einem Stock im Eimer, als könnte er
das Wasser so klären.",
    ],
  });

  while (!tot(held) && !held.loesungswegBrunnen)
  {
    const grabenLabel = held.spurAmBrunnen
```

```

        ? "Dem Ablaufgraben zur Zisterne folgen"
        : held.grovinGenannt
        ? "Nach Grovins altem Bau am Waldrand
suchen"
        : "Den Graben am Brunnenrand verfolgen";
    const items: { id: string; label: string }[]
= [
    { id: "kern", label: "Mit Witwe Kern über
das Wasser sprechen" },
    { id: "dennek", label: "Mit Ratsherr
Dennek sprechen" },
    { id: "brunnen", label: "Den Brunnen
selbst untersuchen" },
    { id: "graben", label: grabenLabel },
    { id: "dorf", label: "Zurück zum
Dorfplatz" },
];
const wahl = await rt.present({
    title: "Trübes Wasser",
    art: "well",
    portrait: null,
    held,
    lines: [
        "Das Wasser im Eimer ist trüb bis auf
den Grund. Es schmeckt nach Eisen, sobald der
Wind vom Wald herüberzieht.",
        held.truebungBestaetigt
        ? "Kern hat die Kranken genannt.
Dennek steht immer noch am Rand, als gehöre der
Brunnen ihm."
        : "Kerns Tür steht einen Spalt offen.
Dennek trommelt mit den Fingern auf die
Brunnenmauer.",
    ],
    choices: items.map((item) => item.label),
});
const id = items[wahl]?.id;

```

```

        if (id === "kern") await kernWasser(rt,
held);
        else if (id === "dennek") await
dennekGespraech(rt, held);
        else if (id === "brunnen") await
brunnenUntersuchen(rt, held);
        else if (id === "graben") await
ablaufgraben(rt, held);
        else return;
    }
}

async function brunnenNachspiel(rt: Runtime,
held: Held) {
    if (held.loesungswegBrunnen === "bestochen") {
        await rt.present({
            title: "Am Brunnen",
            art: "well",
            portrait: null,
            held,
            lines: [
                "Das Wasser ist klarer. Es reicht
trotzdem nicht für alle.",
                "Ein zweiter Eimer bleibt ungefüllt.
Niemand fragt, wohin der Rest läuft.",
            ],
        });
        return;
    }
    await rt.present({
        title: "Am Brunnen",
        art: "well",
        portrait: null,
        held,
        lines: [
            "Der Eimer ist wieder klar bis auf den
Grund.",

```

```

        held.dennekEntlarvt
        ? "Dennek meidet den Brunnenrand. Die
        Finger haben nichts mehr, worauf sie trommeln
        könnten."
        : "Dennek steht noch da. Er rührt nicht
        mehr im Wasser. Er sieht auch nicht zu dir.",
        ],
    });
}

export async function kernWasser(rt: Runtime,
held: Held) {
    if (held.loesungswegBrunnen) {
        await rt.present({
            title: "Bei Witwe Kern",
            art: "apothecary",
            portrait: "kern",
            held,
            lines:
                held.loesungswegBrunnen === "bestochen"
                ? [
                    "Kern wiegt dieselbe Mischung ab,
                    nur seltener.",
                    "„Es reicht länger. Es reicht
                    nicht.“ Sie sieht dich an, als wüsste sie, dass
                    du mehr weißt als das Dorf.",
                ]
                : [
                    "Kern braut zum ersten Mal seit
                    Tagen wieder etwas anderes als Fiebermittel.",
                    "Sie fragt nicht, warum das Wasser
                    klar ist. Sie füllt die Flaschen, solange
                    niemand hustet.",
                ],
        });
        return;
    }
}

```

```

const erst = !held.truebungBestaetigt;
held.truebungBestaetigt = true;
await rt.present({
  title: "Bei Witwe Kern",
  art: "apothecary",
  portrait: "kern",
  held,
  lines: erst
  ? [
    "Kern hat die Ärmel hochgekrempelt.
Auf der Waage liegt dieselbe Kräutermischung,
die nie reicht.",
    "„Bauchschmerzen. Fieber. Ein
metallischer Geschmack. Die Kinder zuerst, dann
die Alten.“",
    "Lohn bietet sie dir nicht an. „Etwas
Fremdes ist im Wasser. Nicht Krankheit allein.
Jemand hat den Brunnen angefasst.“",
    "Unter den Krankenzetteln an der Wand
sind zwei ganz frisch. Beide nennen Häuser, die
nah am Brunnen stehen.",
  ]
  : [
    "Kern wiegt die Mischung neu. Die
Schale senkt sich nicht weit genug.",
    held.grovinGenannt
    ? "„Grovin hat den Brunnen gebaut.
Wenn jemand weiß, wo das Wasser verschwindet,
dann er.“"
    : "„Dennek steht am Rand und rührt,
als könnte ein Stock eine Schuld klären.“",
  ],
});
}

async function dennekGespraech(rt: Runtime,

```

```

held: Held) {
  if (held.dennekEntlarvt) {
    await rt.present({
      title: "Ratsherr Dennek",
      art: "well",
      portrait: null,
      held,
      lines: [
        "Dennek trommelt nicht mehr. Die Finger
        liegen flach auf dem Stein, als müssten sie sich
        festhalten.",
        "„Grovin“, sagt er, als koste ihn der
        Name nichts mehr. „Zisterne am Waldrand. Ich
        habe nicht bezahlt. Das Wasser hat es getan.“",
      ],
    });
    return;
  }

  const lines = [
    "Dennek rührt im Eimer. Das Wasser wird
    dadurch nicht klarer.",
    "„Trockenes Jahr“, sagt er. „Der Brunnen
    gibt, was er kann. Mehr wäre Klage.“",
  ];
  if (held.truebungBestaetigt) {
    lines.push("Du hast Kerns Krankenliste.
    Dennek sieht auf deine Schuhe, nicht auf den
    Eimer.");
  }
  if (held.buergermeisterVertraut) {
    lines.push("Er kennt Holms Vorschuss. Das
    macht ihn höflicher, nicht ehrlicher.");
  }

  const schwer = held.buergermeisterVertraut ||
    held.truebungBestaetigt ? MITTEL : SCHWER;

```

```

const items: { id: string; label: string }[] =
[
  {
    id: "charisma",
    label: held.truebungBestaetigt
      ? "Ihn mit den Kranken stellen
(Charisma, mittel)"
      : "Nach dem trockenen Jahr fragen
(Charisma, schwer)",
    },
  {
    id: "staerke",
    label: held.buergermeisterVertraut
      ? "Ihn im Namen des Rats festlegen
(Stärke, mittel)"
      : "Ihn an die Mauer drücken (Stärke,
mittel)",
    },
  { id: "gehen", label: "Ihn am Eimer lassen"
},
];

const wahl = await rt.present({
  title: "Ratsherr Dennek",
  art: "well",
  portrait: null,
  held,
  lines,
  choices: items.map((item) => item.label),
});
const id = items[wahl]?.id;
if (id === "gehen" || id == null) return;

if (id === "charisma") {
  const ergebnis = probe(held, "Charisma",
held.charisma, schwer, "Denneks Ausflucht
prüfen");

```

```

        await dennekProbe(rt, held, ergebnis);
        return;
    }
    const ergebnis = probe(held, "Stärke",
held.staerke, MITTEL, "Dennek festlegen");
    await dennekProbe(rt, held, ergebnis);
}

async function dennekProbe(rt: Runtime, held:
Held, ergebnis: ReturnType<typeof probe>) {
    if (ergebnis.erfolg) {
        held.dennekEntlarvt = true;
        held.grovinGenannt = true;
        await rt.present({
            held,
            probe: ergebnis,
            lines: [
                "Dennek trommelt einmal zu oft. Dann
bleiben die Finger still.",
                "„Grovin hat den Brunnen gebaut. Wir
haben ihn nicht bezahlt. Seither ist er weg, und
das Wasser geht mit ihm.“",
                "Er lässt den Stock in den Eimer fallen.
Dann sieht er weg, als gehöre der Brunnenrand
nicht mehr zum Rat.",
            ],
        });
        return;
    }
    held.grovinGenannt = true;
    await rt.present({
        held,
        probe: ergebnis,
        lines: [
            "Dennek bleibt stur. „Trockenes Jahr. Mehr
steht nicht im Buch.“",
            "Beim dritten Satz rutscht ihm ein Name:

```

```

Grovin. Er schluckt ihn nicht mehr ganz
hinunter.",
    "Die Finger trommeln weiter. Lügen haben
hier einen Takt.",
    ],
    });
}

async function brunnenUntersuchen(rt: Runtime,
held: Held) {
    if (held.spurAmBrunnen) {
        await rt.present({
            title: "Brunnenschacht",
            art: "well",
            portrait: null,
            held,
            lines: [
                "Der frische Mörtel ist noch weich in
der Fuge.",
                "Der schmale Ablaufgraben führt aus dem
Dorf, gerade genug, dass man ihn für Regen
halten kann.",
            ],
        });
        return;
    }
    const ergebnis = probe(held,
"Geschicklichkeit", held.geschick, LEICHT, "den
Brunnenrand prüfen");
    if (ergebnis.erfolg) {
        held.spurAmBrunnen = true;
        await rt.present({
            title: "Brunnenschacht",
            art: "well",
            portrait: null,
            held,
            probe: ergebnis,

```

```

        lines: [
            "Frischer Mörtel an einer Steinfuge.
Nicht älter als ein paar Nächte.",
            "Dahinter ein schmaler Ablaufgraben,
kaum sichtbar, Richtung Wald.",
            "Denneks Stock hat genau diese Stelle
gemieden.",
        ],
    });
    return;
}
await rt.present({
    title: "Brunnenschacht",
    art: "well",
    portrait: null,
    held,
    probe: ergebnis,
    lines: [
        "Der Stein ist nass. Der Eimer ist trüb.
Mehr gibt der Rand nicht her.",
        "Wer den Graben will, muss ihn später
suchen, im Unterholz, ohne die Fuge als
Zeugin.",
    ],
});
}

async function ablaufgraben(rt: Runtime, held:
Held) {
    if (!held.spurAmBrunnen) {
        const suche = probe(held,
"Geschicklichkeit", held.geschick, MITTEL, "den
Ablaufgraben finden");
        if (!suche.erfolg) {
            await rt.present({
                art: "forest",
                held,

```

```

        probe: suche,
        lines: [
            "Dornen. Nasses Laub. Drei Gräben, die
alle nach Regen aussehen.",
            "Ohne die Fuge am Brunnen bleibt der
Weg eine Behauptung.",
        ],
    });
    return;
}
await rt.present({
    art: "ditch",
    held,
    probe: suche,
    lines: [
        "Du findest den Graben dort, wo das Gras
kürzer ist, als der Regen es erklärt.",
        "Er läuft gerade, zu gerade für
Wildwasser.",
    ],
});
}

await rt.present({
    title: "Ablaufgraben",
    art: "ditch",
    portrait: null,
    held,
    lines: [
        "Der Graben endet an einer halb
überwucherten Zisterne. Das Mauerwerk ist alt.
Die Fugen sind sauber.",
        "Jemand hält sie instand, während das Dorf
hustet.",
    ],
});

```

```

const sneakSchwer = held.spurAmBrunnen ?
MITTEL : SCHWER;
const weg = await rt.present({
  title: "An der Zisterne",
  art: "ditch",
  portrait: null,
  held,
  lines: [
    "Gestrüpp steht dornig vor dem Becken.
Dahinter bewegt sich eine flache Hand über
Wasser, das klarer ist als alles im Dorf.",
  ],
  choices: [
    "Sich durch das Gestrüpp zwingen (Stärke,
leicht)",
    held.spurAmBrunnen
      ? "Sich unbemerkt nähern (Geschick,
mittel)"
      : "Sich unbemerkt nähern (Geschick,
schwer)",
    "Umkehren",
  ],
});
if (weg === 2) return;

let grovinBereit = false;
if (weg === 0) {
  const ergebnis = probe(held, "Stärke",
held.staerke, LEICHT, "durch das Gestrüpp");
  if (!ergebnis.erfolg) {
    const wunde = schaden(held, 1, "Dornen");
    await rt.present({
      held,
      probe: ergebnis,
      log: [wunde],
      lines: [
        "Die Dornen nehmen Stoff und Haut.

```

```

Alarm geben sie nicht.",
    "Du kommst durch, zerrissen, aber
unangekündigt genug.",
    ],
    });
} else {
    await rt.present({
        held,
        probe: ergebnis,
        lines: ["Das Gestrüpp gibt nach, wo
jemand schon öfter durchgegangen ist."],
    });
}
} else {
    const ergebnis = probe(held,
"Geschicklichkeit", held.geschick, sneakSchwer,
"sich der Zisterne nähern");
    if (!ergebnis.erfolg) {
        grovinBereit = true;
        await rt.present({
            held,
            probe: ergebnis,
            lines: [
                "Ein Ast knackt. Die flache Hand
bleibt auf dem Wasser liegen, dann nicht mehr.",
                "Grovin steht auf. In der anderen Hand
eine Grabegabel, deren Zinken blanker sind als
das Werkzeug eines Mannes, der nur Wasser
misst.",
            ],
        });
    }
} else {
    await rt.present({
        art: "sneak",
        held,
        probe: ergebnis,
        lines: [

```

```

        "Du kommst seitlich an das Becken.
Grovin prüft den Stand noch mit der flachen
Hand, als gehöre ihm die Ruhe.",
        ],
        });
    }
}

    if (tot(held)) {
        held.todesort = "zisterne";
        return;
    }
    await grovinZisterne(rt, held, grovinBereit);
}

async function grovinZisterne(rt: Runtime, held:
Held, bewaffnet: boolean) {
    const lines = [
        "Das Wasser in der Zisterne ist klar bis auf
den Grund. Grovin sieht zuerst darauf, dann auf
dich.",
        "„Ich habe dem Dorf diesen Brunnen gebaut,
und es hat mich nicht bezahlt. Also nimmt das
Wasser, was mir zusteht.“",
    ];
    if (bewaffnet) {
        lines.push("Die Grabegabel bleibt zwischen
euch. Ihre Zinken tropfen. Nicht von Regen.");
    }
    if (held.dennekEntlarvt) {
        lines.push("„Dennek trommelt, wenn er lügt.
Ich habe das früher gehört als du.“");
    }

    await rt.present({
        title: "Grovin's Zisterne",
        art: "ditch",
    });
}

```

```

    portrait: null,
    held,
    lines,
  });

  if (!held.grovinsGrund) {
    const frage = await rt.present({
      held,
      lines: ["Grovin wartet. Nicht lange."],
      choices: [
        "Ihn nach der ausgebliebenen
Entschädigung fragen",
        "Sofort handeln",
      ],
    });
    if (frage === 0) {
      held.grovinsGrund = true;
      held.grovinGenannt = true;
      await rt.present({
        held,
        lines: [
          "Grovin legt die Hand wieder aufs
Wasser, als könnte er daran messen, ob du
zuhörst.",
          "„Drei Jahre Arbeit. Kein Lohn. Dennek
hat gesagt, das Amt zahle später. Später ist ein
Grab ohne Stein.“",
          "Er will nicht das Dorf vergiften. Er
will, dass jemand endlich die Rechnung liest.",
        ],
      });
    }
  }

  const kampfSchwer = bewaffnet ? SCHWER :
MITTEL;
  const oeffnenSchwer = held.spurAmBrunnen ?

```

```

MITTEL : SCHWER;
const items: { id: string; label: string }[] =
[
  {
    id: "zerstoeren",
    label: bewaffnet
      ? "Die Sperre gewaltsam brechen (Stärke,
schwer)"
      : "Die Sperre gewaltsam brechen (Stärke,
mittel)",
    },
  {
    id: "oeffnen",
    label: held.spurAmBrunnen
      ? "Die Sperre unbemerkt umlegen
(Geschick, mittel)"
      : "Die Sperre unbemerkt umlegen
(Geschick, schwer)",
    },
];
if (held.grovinsGrund) {
  items.push({ id: "handeln", label: "Ihm
Holms Entschädigung versprechen (Charisma,
mittel)" });
}
items.push({
  id: "besteichen",
  label: held.gold >= 5 ? "Ihn mit 5 Gold
ruhigstellen" : "Ihn mit Gold ruhigstellen – zu
wenig Gold",
});
items.push({ id: "gehen", label: "Die Zisterne
verlassen" });

const wahl = await rt.present({
  title: "Grovins Zisterne",
  art: "ditch",

```

```

    portrait: null,
    held,
    lines: ["Was tust du?"],
    choices: items.map((item) => item.label),
  });
  const id = items[wahl]?.id;
  if (id === "gehen" || id == null) return;

  if (id === "bestechen") {
    if (held.gold < 5) {
      await rt.present({
        held,
        lines: ["Grovin sieht in deinen Beutel,
ohne ihn zu berühren. „Später ist schon einmal
gekommen. Es war leer.“"],
      });
      return;
    }
    held.gold -= 5;
    held.loesungswegBrunnen = "bestochen";
    await rt.present({
      held,
      log: ["→ 5 Gold. Grovin behält seine
Zisterne."],
      lines: [
        "Grovin nimmt das Gold, ohne zu zählen.
Er kennt den Betrag, den man zahlt, damit
niemand fragt.",
        "Ein Teil des Wassers läuft zurück. Ein
Teil bleibt hier. Das Dorf wird seltener husten
und nie wirklich satt trinken.",
      ],
    });
    await brunnenEnde(rt, held);
    return;
  }

```

```

    if (id === "handeln") {
        const ergebnis = probe(held, "Charisma",
held.charisma, MITTEL, "Grovin ein Versprechen
geben");
        if (ergebnis.erfolg) {
            held.loesungswegBrunnen = "verhandelt";
            held.grovinVersprechen = true;
            await rt.present({
                held,
                probe: ergebnis,
                lines: [
                    "Grovin hört das Wort Amt, ohne
wegzusehen.",
                    "„Wenn Holm zahlt, öffne ich selbst.
Wenn er nicht zahlt, kommt das Wasser nicht
zurück. Das ist kein Drohen. Das ist die alte
Rechnung.“",
                    "Er legt die Sperre um. Das klare
Wasser läuft den Graben zurück, als hätte es den
Weg nie vergessen.",
                ],
            });
            await brunnenEnde(rt, held);
            return;
        }
        await rt.present({
            held,
            probe: ergebnis,
            lines: [
                "Grovin schüttelt den Kopf. „Versprechen
habe ich schon. Sie wiegen weniger als diese
Hand auf dem Wasser.“",
            ],
        });
        return;
    }
}

```

```

    if (id === "oeffnen") {
        const ergebnis = probe(held,
            "Geschicklichkeit", held.geschick,
            oeffnenSchwer, "die Wassersperre umlegen");
        if (ergebnis.erfolg) {
            held.loesungswegBrunnen = "geoeffnet";
            await rt.present({
                art: "sneak",
                held,
                probe: ergebnis,
                lines: [
                    "Die Sperre geht leise. Grovin merkt
es erst, als der Stand unter seiner Hand
sinkt.",
                    "Er flucht ohne Stimme. Das Dorf wird
nie erfahren, warum der Eimer morgen klar ist.",
                ],
            });
            await brunnenEnde(rt, held);
            return;
        }
        const wunde = schaden(held, bewaffnet ? 4 :
            2, "Grabegabel");
        held.todesort = tot(held) ? "zisterne" :
            held.todesort;
        await rt.present({
            art: tot(held) ? "death" : "combat",
            held,
            probe: ergebnis,
            log: [wunde],
            lines: tot(held)
                ? [
                    "Die Zisterne bleibt klar und
still.",
                    "Das Dorf wartet weiter auf einen
Boten, der nicht zurueckkommt.",
                ]
            : []
        });
    }

```

```

        : ["Grovin ist schneller am Hebel als
du. Die Gabel findet Stoff, dann Haut."],
    });
    if (tot(held)) return;
    await vielleichtHeiltrank(rt, held);
    return;
}

const ergebnis = probe(held, "Stärke",
held.staerke, kampfSchwer, "die Umleitung
zerstören");
if (ergebnis.erfolg) {
    held.loesungswegBrunnen = "zerstoert";
    held.grovinGeflohen = true;
    await rt.present({
        art: "combat",
        held,
        probe: ergebnis,
        lines: [
            "Das Holz der Sperre bricht. Wasser
schießt den Graben zurück, braun vor
aufgewühltem Grund, dann klarer.",
            "Grovin flieht, bevor du die Gabel ganz
siehst. Der Waldrand nimmt ihn, ohne zu
fragen.",
        ],
    });
    await brunnenEnde(rt, held);
    return;
}

const wunde = schaden(held, bewaffnet ? 4 : 3,
"Kampf an der Zisterne");
held.todesort = tot(held) ? "zisterne" :
held.todesort;
await rt.present({
    art: tot(held) ? "death" : "combat",
    held,

```

```

        probe: ergebnis,
        log: [wunde],
        lines: tot(held)
        ? [
            "Du gehst in das klare Wasser und
bleibst dort.",
            "Die Zisterne behält den Stand. Das
Dorf behält den Husten.",
        ]
        : ["Die Sperre hält. Grovin auch. Du musst
später wiederkommen oder anders fragen."],
    });
    if (tot(held)) return;
    await vielleichtHeiltrank(rt, held);
}

async function brunnenEnde(rt: Runtime, held:
Held) {
    if (held.loesungswegBrunnen === "bestochen") {
        await rt.present({
            title: "Zwei Brunnen, ein Dorf",
            art: "well",
            portrait: "kern",
            held,
            lines: [
                "Das Wasser wird klarer, aber nie
wirklich genug für alle.",
                "Kern braut weiter dieselbe Mischung,
nur seltener.",
                "Du trägst das Wissen allein.",
            ],
        });
        return;
    }

    if (held.loesungswegBrunnen === "zerstoert" &&
!held.dennekEntlarvt) {

```

```

    await rt.present({
      title: "Wasser mit einem Riss",
      art: "well",
      portrait: null,
      held,
      lines: [
        "Das Wasser fließt wieder, aber Grovin
ist verschwunden, nicht verschwunden genug.",
        "Manche Nächte hört man Schritte am
Waldrand, die niemand dem Dorf zuordnen will.",
      ],
    });
    return;
  }

  const log: string[] = [];
  if (held.truebungBestaetigt &&
!held.inventar.includes(HEILTRANK)) {
    log.push(nimm(held, HEILTRANK));
  } else if (held.truebungBestaetigt) {
    log.push(goldPlus(held, 2, "Kerns Dank, weil
die Flaschen reichen"));
  }

  await rt.present({
    title: "Klares Wasser",
    art: "well",
    portrait: "kern",
    held,
    log: log.length ? log : undefined,
    lines: [
      "Am nächsten Morgen ist der Eimer am
Brunnen wieder klar bis auf den Grund.",
      "Kern braut zum ersten Mal seit Tagen
wieder etwas anderes als Fiebermittel.",
      held.dennekEntlarvt
        ? "Niemand im Dorf fragt, warum – nur

```

```
Dennek meidet für eine Weile den Brunnenrand."  
    : "Niemand im Dorf fragt, warum. Fragen  
kosten hier Kraft, die man zum Trinken  
braucht.",  
    held.grovinVersprechen  
    ? "Holm schuldet Grovin jetzt eine Zahl,  
die nicht in der Kasse steht. Du hast es  
versprochen."  
    : "Das Wasser schmeckt nach Stein, nicht  
nach Metall. Das reicht für ein Tal.",  
    ],  
    });  
}
```